

# Programación Dinámica

Análisis y Diseño de Algoritmos / Ingeniería Civil Informática  
Departamento Ciencias de la Computación y  
Tecnologías de la Información

Universidad del Bío-Bío

Profesor: Gilberto Gutiérrez R.

## 1 Distancia editada entre dos Strings

Sean  $u$  y  $v$  dos cadenas de caracteres. Se desea transformar  $u$  en  $v$  con el mínimo número de operaciones básicas del tipo siguiente: eliminar un carácter, añadir un carácter y cambiar un carácter. Por ejemplo, podemos pasar de *abbac* a *abcba* en tres pasos:

1. *abbac*  $\rightarrow$  *abac* (eliminamos b en la posición 3)
2. *ababc* (añadimos b en la posición 4)
3. *abcba* (cambiamos a en la posición 3 por c)

Sin embargo, esta transformación no es óptima. Lo que queremos en este caso es diseñar un algoritmo que calcule el número mínimo de operaciones, de esos tres tipos, necesarias para transformar  $u$  en  $v$  y cuáles son esas operaciones, estudiando su complejidad en función de las longitudes de  $u$  y  $v$ . En primer lugar, la transformación mostrada arriba no es óptima ya que podemos pasar de *abbac* a *abcba* en sólo dos pasos: *abbac*  $\rightarrow$  *abcac* (cambiamos b en la posición 3 por c) y luego  $\rightarrow$  *abcba* (cambiamos a en la posición 4 por b).

Sea  $m$  el tamaño de  $u$  y  $n$  el tamaño de  $v$  y  $OB(m, n)$  es el número de operaciones básicas para transformar  $u$  en  $v$ . Podemos fijarnos en el último elemento de  $u$  y en el último de  $v$ .

$$OB(m, n) = OB(m - 1, n - 1), \text{ si } u_m = v_n$$

Pero si los últimos elementos fueran distintos habría que elegir la situación más beneficiosa de entre las siguientes:

1. Considerar la primera cadena tal cual y la segunda sin el último elemento
2. Considerar la segunda cadena tal cual y la primera sin el último elemento
3. Considerar la primera cadena y la segunda sin el último elemento

es decir,

$$OB(m, n) = 1 + \text{Min}\{OB(m, n-1), OB(m-1, n), OB(m-1, n-1)\} \text{ si } u_m \neq v_n, m \neq 0, n \neq 0$$

Las condiciones de borde son las siguientes:

$$OB(0, 0) = 0, OB(m, 0) = m, OB(0, n) = n$$

Notar que la implementación recursiva es inmediata a partir de la definición. Pero que ocurre con el rendimiento de este algoritmo recursivo. Plantee la ecuación de recurrencia, resuélvala y obtenga la complejidad del algoritmo.

Siguiendo la misma estrategia para encontrar un algoritmo eficiente (visto en la clase pasada) para el problema del cálculo del coeficiente binomial, encuentre un algoritmo  $O(m \cdot n)$  para resolver el problema. Le recuerdo que la idea es resolver problemas pequeños, luego combinarlos para resolver problemas más grande y que se acerquen al problema original.